

Names: Dave Burgoin, Moyo Ogunjobi, Jacob Plourde

Group #: 6

Course: ENSF 460 - Embedded Software and Hardware Systems

Assignment Number: Assignment 3 Driver Project

Date Submitted: October 09, 2026

1 Flow Chart

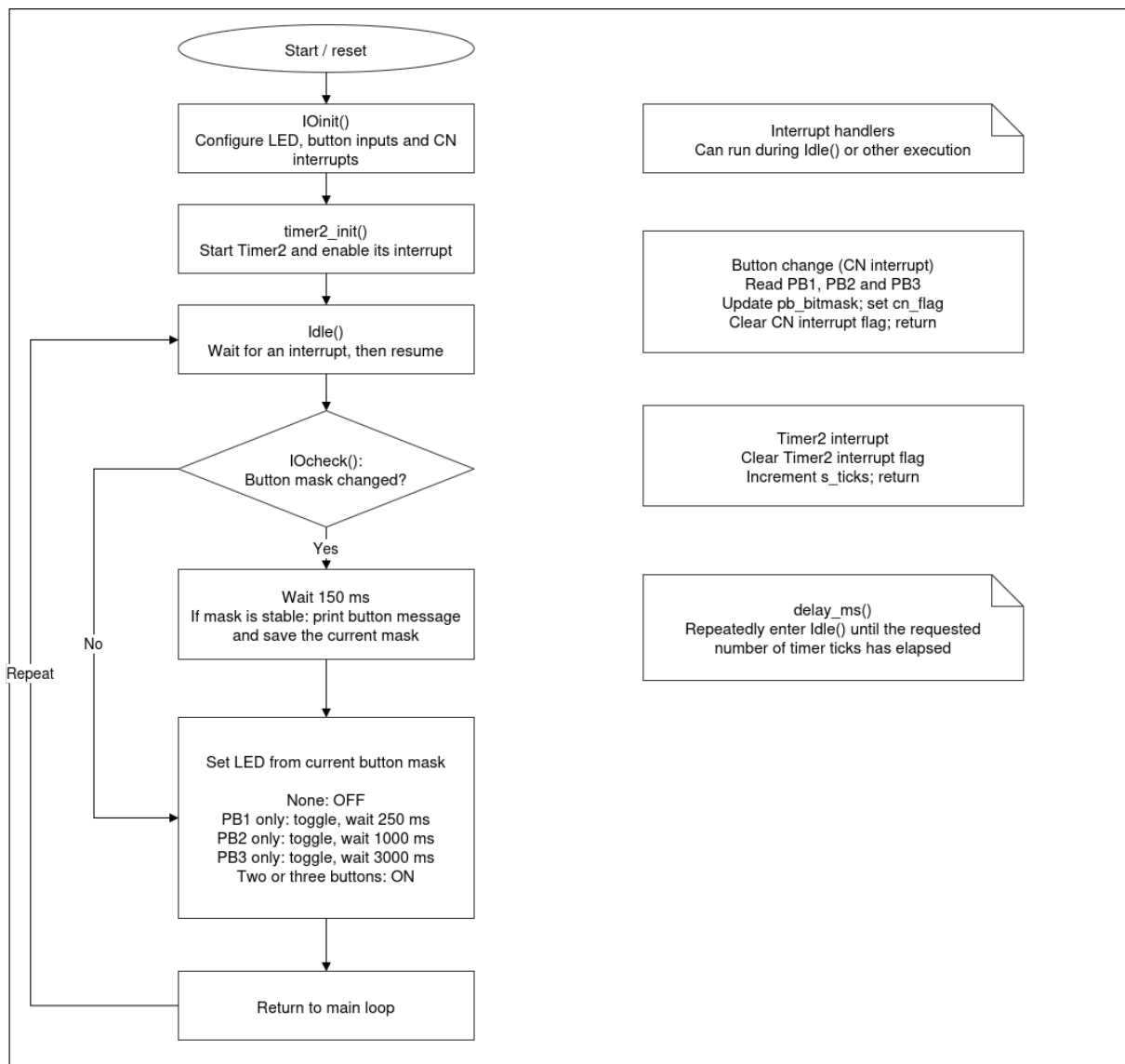


Figure 1: Flow chart for our program

1.1 Startup

The program begins at the entry point of `main()`, where the I/O and timer initialization functions are called. These functions are reused from Labs 1 and 2 and are explained in the previous lab reports.

The program then enters its main superloop, which contains the power-saving `Idle()` call and the `IOcheck()` function (defined in `IOs.h`). When `Idle()` is executed, the CPU suspends instruction execution while the peripherals continue operating. An interrupt from either Timer2 or the Change Notification (CN) module wakes the CPU, causing the corresponding ISR to execute. Once the ISR completes, execution resumes at the instruction immediately following `Idle()`, which allows the program to continue into `IOcheck()`.

1.2 Push Button Debouncing and State Detection

The program compares the current push-button bit field (a `volatile` variable updated directly by the CN ISR) against the previously accepted push-button state.

If the values differ, the program enters a debounce loop and takes a snapshot of the current push-button bit field. It then waits 150 ms before comparing the snapshot against the current bit field.

If the values match, the button state is considered stable. The program prints the corresponding message to the terminal using the bit field value as an index into the messages array, which contains constant pointers to predefined strings.

1.3 UART

On the first call of `printf()`, the UART2 interface is lazily initialized with a `uart_init(2, 4800)` call. First, the BRGL value is calculated using the formula found in the data sheet, based on the current clock frequency:

$$UxBRG = \frac{F_{CY}}{16 \times (\text{Baud Rate})} - 1$$

This ensures that the baud rate generator has the correct value for a baud rate of 4800.

The UART interface is then configured with a standard 8 data bits, no parity, and 1 stop bit.

When a button is pressed, using the same `pb_bitmask`, we index into an array with the available messages, and it uses `printf()` to print the message.

`printf()` can be used because we override the `write()` syscall with our own UART implementation, which first checks if the transmission buffer is ready to receive a new byte, then queues the data out, and waits for the transmission to complete.

1.4 LED Control

After the debounce check, the program enters a switch statement that controls the LED based on the current push-button bit field.

State 0 (No PB pressed): The LED is turned OFF and remains OFF until the push-button state changes.

State 1 (PB1 pressed): The LED toggles between ON and OFF, followed by a call to `delay_ms(250)` where 250 is the amount of time in ms to block.

State 2 (PB2 pressed): The LED toggles between ON and OFF, followed by a call to `delay_ms(1000)` where 1000 is the amount of time in ms to block.

State 4 (PB3 pressed): The LED toggles between ON and OFF, followed by a call to `delay_ms(3000)` where 3000 is the amount of time in ms to block.

States 3, 5, 6, and 7 (Multiple PBs pressed): The LED is turned ON and remains ON while multiple push buttons are pressed.

For states 1, 2, and 4, each LED toggle inverts its current output state (ON becomes OFF and vice versa). The `delay_ms(x)` function provides a delay of x milliseconds before the next toggle can occur.

1.5 Return to Main Superloop

Once the switch statement has been evaluated, `I0Check()` returns control to `main()`. The processor executes `Idle()` again and remains idle until another interrupt occurs.

Since Timer2 generates an interrupt every 1 ms, the CPU normally wakes and executes `IOCheck()` approximately every 1 ms when no blocking delays are active. A CN interrupt can also wake the CPU between Timer2 interrupts. If the program is executing a delay when an interrupt occurs, it resumes the interrupted operation rather than immediately restarting the superloop.

1.6 Timer2 Interrupt Service Routine

The Timer2 ISR executes every 1 ms and increments a global system tick counter (`systick`).

This counter provides the time reference for the `delay_ms()` function. Rather than using a software busy-wait loop, the function compares the elapsed time against a requested delay using the following logic:

$$(\text{systick} - \text{previous_systick}) \geq \text{delay}$$

Once the elapsed time reaches the requested delay, the function returns and program execution continues.

Handling the timer this way guarantees precision, while still being power-efficient. A logic analyzer shows that while a `delay_ms()` call is running, there is only a sub-2% duty cycle for CPU activity, meaning the CPU is idled for a significant majority of the time. Since the overwhelming amount of power consumed is coming from timers and peripherals the overall change in power consumption between idle and running is ~90uA meaning on a 2% duty cycle, the change in power consumed is only in the order of ~2uA. When compared to the total idle consumption of ~400uA, the systick overhead current is negligible for a tick of 1ms.

1.7 Change Notification Interrupt Service Routine

The CN ISR monitors the push buttons connected to RA4, RB4, and RB7.

Change Notification interrupts are triggered by changes on the enabled inputs, allowing the program to detect both rising and falling edges.

When a CN interrupt occurs, the ISR reads `PORTA` and `PORTB` and applies the required bit masks and shifts to construct a three-bit push-button control field.

Each bit represents one push button:

Bit 0: PB1 (RB7)

Bit 1: PB2 (RB4)

Bit 2: PB3 (RA4)

The resulting bit field is stored in a global `volatile` variable, allowing `IOCheck()` to evaluate the current push-button state after the ISR completes.

The ISR then clears the CN interrupt flag and returns control to the interrupted program.

2 Report Questions

2.1 What baud rate is used, and how do you know?

The terminal baud rate is 4800 baud. Calls `newC1k(500)`, setting `FOSC` to 500,000 Hz, so $F_{CY} = \frac{F_{OSC}}{2} = 250,000 \text{ Hz}$. In `UART2.c`, `2MODE = 0x0008` sets `BRGH = 1`, and `InitUART2()` selects `U2BRG = 12` for this clock.

$$\text{Baud} = \frac{F_{\text{CY}}}{4 \times (\text{U2BRG} + 1)}$$

$$\text{Baud} = \frac{250000}{4 \times (12 + 1)}$$

Baud = 4807.69 baud, approximately 4800 baud

2.2 Why does “PB3 event” appear twice for each button click?

A click includes a press and a release. Each changes the input level and triggers the CN interrupt. The ISR sets `PB3_event = 1`, and the main loop prints the message and clears the event flag. The original ISR does not identify which button changed, so any a enabled button can produce the same message. Normally this gives one message for the press and another for the release.

2.3 Does the CN ISR keep executing while a button is held?

No. CN interrupts respond to changes in the input level. Holding a button steadily does not create further changes, so it does not repeatedly trigger the ISR. Releasing it, changing another enabled button, or contact bounce can trigger another CN interrupt.